

Gradle: Making Software Development Easier

Gradle is a build tool, which accelerates productivity. As the blurb on its website says, “From mobile apps to micro services, from small startups to big enterprises, Gradle helps teams build, automate and deliver better software, faster.”



Gradle is an open source build system which is familiar to most Android developers. It uses a DSL (Domain Specific Language) built on the Groovy language for configurations, while Direct Acyclic Graph is used to determine the order of its tasks. This article explores Gradle's features that make software development on Android easier.

Separating logic and resources for flavours or build variants

When developing projects, you may want to develop a mock project that uses sample data or a library like *Stetho* for network inspection. Consider an example where *Application* class initialises the needed libraries. Here, we can create a class named *AppController*, which has a method *attach* (*context*) (refer to Figure 1).

```
// check the file path in comments
// app/src/debug/java/com/example/appname/AppController.
java
import android.app.Application;
import com.facebook.stetho.Stetho;
```

```
public class AppController {

    public static void attach(Application application) {
        Stetho.initializeWithDefaults(application);
    }
}

// app/src/release/java/packagename/AppController.java

public class AppController {

    public static void attach(Application application) {
        // analytics sdk
    }
}
```

Check the file path in the comments. On building the project, if it is a *debug* build, Gradle uses class files from the *debug* folder, by default, and from the *main* folder for common modules. In the *application* class (in the *main* folder), you can just use *Appcontroller* class methods.